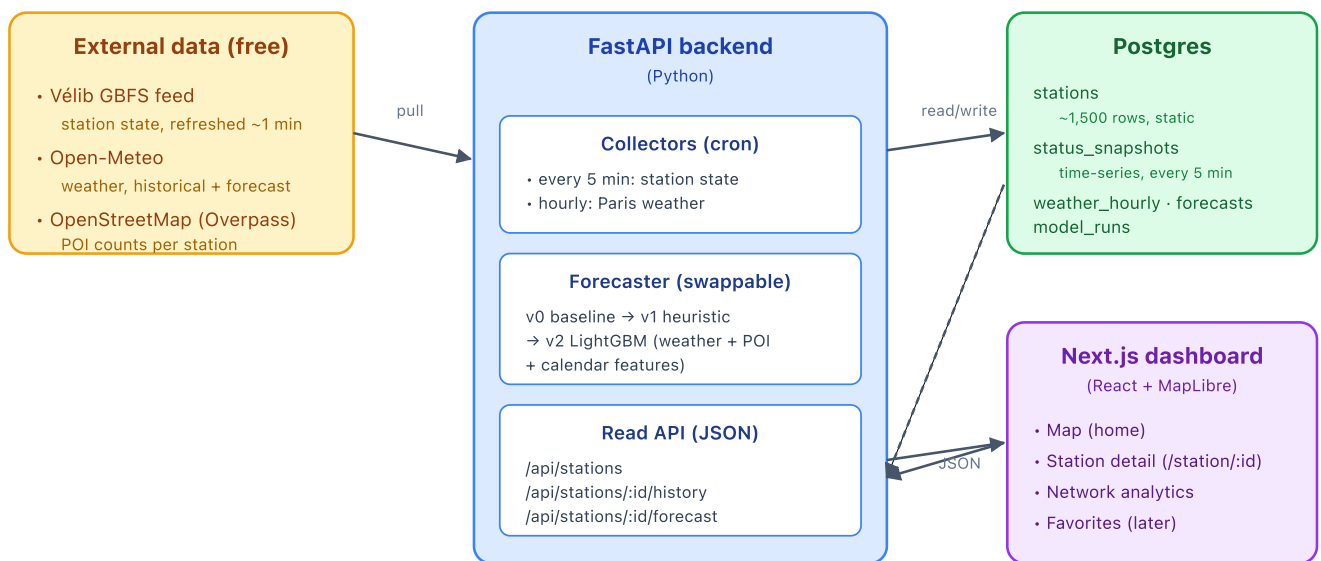


Vélib Wizard

A live Paris Vélib' dashboard with a 2-hour station-fullness forecast.

What it does. Shows every Vélib station on a Paris map, color-coded by how full it currently is, and predicts where each station will be in ~2 hours. The 2h forecast is the centerpiece — the dashboard exists to make it visible and useful.

1 · The big picture



Three boxes that matter, one direction of dependency. The backend is the only thing that talks to the outside world. The dashboard talks only to the backend. Postgres is the single source of truth for everything time-series related.

2 · Data flow, end to end

EVERY 5 MIN

1. Collect

A cron job inside the backend hits the Vélib GBFS feed and inserts one snapshot per station (~1,500 rows) into `status_snapshots`. Weather lands hourly into `weather_hourly`.

EVERY 10 MIN

2. Forecast

The forecaster reads the latest snapshots + features, runs inference for all stations in one batched call, and writes the predicted 2h-out fullness into `forecasts`. Reads are now $O(1)$.

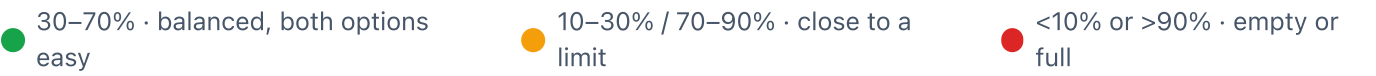
ON DEMAND

3. Serve

When you open the dashboard, it calls the read API. The backend joins the latest snapshot + the cached forecast and returns JSON. The map draws ~1,500 dots colored by current fullness.

3 · What the user sees

Each station gets one number: **how full is it, 0%–100%** (= bikes ÷ capacity). Color is bipolar — green when the station is balanced, red at either extreme — so the same dot tells you both "can I find a bike here?" and "can I find a dock here?":



Three views ship in this order:

Page	What it shows
<code>/</code> · Map	Paris with all ~1,500 stations as colored dots. Click → side panel with current fullness bar + bikes/docks/capacity, link into the detail page.
<code>/station/:id</code> · Detail	Header with current fullness, last 24h occupancy chart, and (M3) a 2h forecast curve on top of it. (Later) POI context — what's near this station.
<code>/network</code> · Analytics	City-wide tiles (% empty / % full right now), a Paris-wide hourly heatmap (M5), plus the "About the model" panel comparing v0 / v1 / v2 accuracy.

4 · How the forecast actually works

The forecasting code has a fixed interface — `predict(station_id, horizon=120) → fullness` — and three swappable implementations that get more capable as more historical data piles up. This is the centerpiece of the project and the strongest narrative for a portfolio piece: "here's how accuracy improved from v0 to v2."

Generation	Needs	How it predicts
v0 — Naïve	No history	"In 2h the station will be roughly where it is now." Bad on its own, just unblocks the UI so the dashboard isn't waiting on data.
v1 — Heuristic	~2 weeks of data	For each station, compute the average fullness at each (<i>day-of-week</i> × <i>hour</i>) bucket. Predict the historical average for the target hour, blended with current state. Pure SQL + numpy.
v2 — LightGBM	~4–6 weeks of data (or an archive backfill)	One global gradient-boosted model trained on a rolling 60-day window. Features below. Retrained nightly. Honest accuracy comes from a held-out last-7-days test split.

Features used by v2

Station context • Lat / lon / capacity • POI counts within 300m: universities, offices, schools, metros, restaurants, parks (one-shot from OSM) • Cluster ID (commuter / student / leisure / tourist) derived from each station's daily usage fingerprint	Recent dynamics • Current fullness • Fullness lags at -15 / -30 / -60 min • Net flow over the last 60 min (bikes arrived minus bikes left)	Time + weather • Hour, day-of-week, is_weekend, month • French public holidays, Paris school holidays (zone C) • Temperature, precipitation, wind — using forecast values for the target horizon
--	---	---

This is what makes the project "more than a viz". The user's idea — students near universities, office workers near offices — is exactly what these features encode.

5 · The build, in five phases

Phase	Outcome	Status
M1	Skeleton + GBFS collector	● Done. Monorepo, FastAPI, Postgres, Next.js scaffold, 1,511 stations flowing in every 5 min.

M2	Live dashboard (no forecast yet)	● Done. Map with fullness coloring, side panel, station detail with 24h history, network summary page.
M3	Forecast v1 — the heuristic	~2 days. Adds the day-of-week × hour heuristic + a forecast endpoint + a forecast curve on the detail chart. Becomes meaningfully predictive after ~2 weeks of accumulated data.
M4	Features + LightGBM v2	~4–5 days. POI ingest, weather collector, calendar features, LightGBM model, nightly retrain, accuracy metrics surfaced in the UI.
M5	Polish + analytics + deploy	~2–3 days. Favorites view, network analytics page, model-comparison chart, deploy to Vercel + Fly.io / Railway + Supabase / Neon, write the portfolio case study.

6 • Things to know about cost & hosting

Everything fits inside free tiers: Vercel for the dashboard, Fly.io or Railway free tier for the backend (one tiny container running FastAPI + the in-process scheduler), Supabase or Neon Postgres free tier. External data (Vélib GBFS, Open-Meteo, OpenStreetMap Overpass, Carto basemap) is free with no API key required. The single deliberate cost ceiling is Postgres storage — status snapshots get downsampled from 5-minute raw to hourly aggregates after 30 days, keeping us comfortably under 500 MB.